



۱. مقدمه و هدف فاز دوم

در فاز اول، هسته‌ی اصلی بازی **Doodle Jump** را ساختید: حرکت بازیکن، فیزیک گرانش و پرش، اسکرول بی‌پایان، و سلسله‌مراتب سکوها (عادی، متحرک، شکستنی) به همراه فنر. آن پیاده‌سازی باید همچنان به عنوان پایه‌ی این فاز باقی بماند – هیچ‌یک از رفتارهای پذیرفته‌شده‌ی فاز اول نباید در این فاز نقض شوند.

در فاز دوم، پنج قابلیت جدید به بازی اضافه می‌کنید:

۱. هیولاها (Monsters): موجودات متحرک خطرناک که هم می‌توانند بازیکن را از پا دریاورند و هم می‌توانند تخت پرش باشند.
۲. شلیک کاراکتر اصلی (Shooting): توانایی بازیکن برای شلیک گلوله به سمت هیولاها.
۳. صدا (Sound): موسیقی پس‌زمینه و جلوه‌های صوتی.
۴. منوی تنظیمات (Settings): صفحه‌ای جدید برای کنترل صدا و انتخاب سطح دشواری بازی.
۵. حفره‌ها (Holes): تله‌های مکش که بازیکن را به درون خود می‌کشند.

⚠ توسعه، نه بازنویسی

این فاز باید روی همان معماری شی‌گرای فاز اول بنا شود. اگر برای اضافه‌کردن این قابلیت‌ها مجبور به بازنویسی کامل کلاس‌های فاز اول شدید، احتمالاً طراحی اولیه به اندازه‌ی کافی قابل توسعه نبوده است – نمره‌ی این بخش بر اساس میزان هم‌راستایی معماری جدید با معماری فاز اول نیز سنجیده می‌شود.

سازگاری

همه‌ی قابلیت‌های زیر باید به‌طور هم‌زمان و سازگار با یکدیگر در بازی حضور داشته باشند؛ برای مثال، بازیکن باید بتواند هم‌زمان که در حال سقوط از یک سکوی شکستنی است، به یک هیولا شلیک کند، و اگر در همین لحظه با یک حفره برخورد کند، رفتار حفره باید بر همه‌چیز دیگر اولویت داشته باشد (جزئیات کامل در بخش‌های بعدی آمده است).



۲. هیولاها (Monsters)

هیولاها موانع متحرک جدیدی هستند که هم‌زمان با سکوها در دنیای بازی تولید می‌شوند. برخلاف سکوها، برخورد با هیولا می‌تواند هم مفید باشد (اگر از بالا فرود بیایید) و هم کشنده (اگر از هر جهت دیگری با آن برخورد کنید).

۲.۱ ظاهر و حرکت

- دست‌کم دو گونه‌ی ظاهری متفاوت برای هیولا در نظر بگیرید (تصاویر آماده در `assets/` موجود است) و در هنگام تولید، به‌صورت تصادفی یکی را انتخاب کنید.
- هیولاها، در کنار برخی سکوها، تازه‌تولیدشده و با احتمال محدود ظاهر می‌شوند.
- هیولا به‌طور افقی حرکت می‌کند و در برخورد با لبه‌های صفحه، جهت حرکتش را برعکس می‌کند – دقیقاً مشابه رفتار سکوی متحرک.

۲.۲ قوانین تولید

- موقعیت تولید هیولا باید طوری انتخاب شود که با هیچ سکو، هیولا یا حفره‌ی دیگری هم‌پوشانی یا تماس نداشته باشد.
- هیولا هرگز نباید تنها مسیر رسیدن به سکوی بعدی را مسدود کند؛ بازیکن همیشه باید بتواند از کنار آن رد شود.

۲.۳ برخورد با بازیکن

برخورد بازیکن با هیولا باید بر اساس جهت برخورد، دو نتیجه‌ی کاملاً متفاوت داشته باشد:

شرایط برخورد	نتیجه
بازیکن در حال سقوط است و دقیقاً روی سطح بالایی هیولا فرود می‌آید	یک پرش قوی به بازیکن اعمال می‌شود.
هر نوع برخورد دیگر (از پهلو یا از پایین)	پایان فوری بازی.

۲.۴ سلامتی و بازتولید

- هر هیولا دارای مقدار مشخصی سلامتی (**health**) است که با شلیک گلوله به آن کاهش می‌یابد (به بخش ۳ مراجعه کنید). با رسیدن سلامتی به صفر، هیولا از بین می‌رود.
- سلامتی اولیه‌ی هیولا باید به سطح دشواری انتخاب‌شده وابسته باشد (سطوح سخت‌تر، هیولاها، بادوام‌تر دارد).
- هیولایی که از بین می‌رود یا از پایین صفحه خارج می‌شود، باید بازتولید شود (در موقعیت جدیدی بالاتر از بازیکن) تا تعداد اشیاء فعال محدود و مصرف حافظه پایدار بماند.



۳. شلیکِ کاراکترِ اصلی (Shooting)

بازیکن اکنون می‌تواند برای دفاع از خود، به سمت هیولاها شلیک کند. این قابلیت باید کاملاً مستقل از حرکت چپ/راست و پرشِ خودکار عمل کند – بازیکن باید بتواند هم‌زمان حرکت کند و شلیک کند.

۳.۱ حالتِ شلیک

عملکرد	کلید
ورود به حالتِ شلیک و شلیک توپ به صورت پیوسته با یک نرخ ثابت	Space (نگه‌داشته‌شده)

- تا زمانی که کلید نگه‌داشته شده، بازیکن باید در حالتِ شلیک نمایش داده شود (پوز متفاوتی نسبت به حالتِ عادی دارد – تصاویر آماده در `assets/` موجود است).
- با رهاکردنِ کلید، بازیکن بلافاصله به پوزِ عادی بازمی‌گردد.

۳.۲ گلوله‌ها

- هر گلوله از یک نقطه‌ی ثابت روی بدنِ بازیکن (نوکِ خرطوم) تولید می‌شود و با سرعتِ ثابت به سمتِ بالا حرکت می‌کند.
- گلوله‌ها فقط با هیولاها برهم‌کنش دارند؛ باید بدونِ هیچ اثری از میانِ سکوها، فترها و حفره‌ها عبور کنند.
- گلوله‌ای که با هیولا برخورد می‌کند، بلافاصله حذف شده و یک واحد از سلامتی آن هیولا کم می‌شود.
- گلوله‌ای که از بالای صفحه خارج شود (به هیچ هیولایی برخورد نکند) نیز باید حذف شود.

⚠ مدیریتِ حافظه‌ی گلوله‌ها

گلوله‌ها به‌طور مداوم در حال تولید هستند. هر گلوله‌ای که از صفحه خارج شود یا به هدف برخورد باید بلافاصله با `delete` آزاد و از لیستِ اشیاءِ فعال حذف شود؛ در غیر این صورت حافظه به سرعت انباشته می‌شود.

۳.۳ نرخِ آتش وابسته به دشواری بازی

فاصله‌ی زمانی بین دو شلیکِ متوالی باید به سطحِ دشواری بازی بستگی داشته باشد:

سطحِ دشواری	نرخِ آتش
Easy	سریع‌تر
Medium / Hard	کندتر

مقادیرِ دقیقِ زمانی به عهده‌ی خودتان است؛ فقط باید تفاوتِ محسوسی میانِ سطوح وجود داشته باشد.



۴. صدا (Sound)

بازی باید دارای یک موسیقی پس‌زمینه و چند جلوه‌ی صوتی کوتاه برای رویدادهای کلیدی باشد (فایل‌های صوتی هر بخش در `assets/` موجود است)

۴.۱ موسیقی پس‌زمینه

- موسیقی باید تنها در وضعیت‌های **منوی اصلی** و **تنظیمات** پخش شود.
- با شروع گیم‌پلی، موسیقی باید متوقف شود؛ با بازگشت به منو یا رفتن به صفحه‌ی تنظیمات، دوباره پخش شود.

۴.۲ جلوه‌های صوتی

رویداد	زمان پخش
پرش از سکو	هر بار که بازیکن با فرود موفق روی یک سکوی عادی/متحرک پرش می‌کند
باخت	لحظه‌ای که بازی به وضعیت پایان می‌رود
شلیک	هر بار که یک گلوله‌ی جدید تولید می‌شود

🔗 بازاستفاده از الگوی جنریک فاز اول

در فاز اول از شما خواسته شد یک `ResourceManager<T>` جنریک برای بارگذاری یک‌باره و کش منابع (مثل `sf::Texture`) بسازید. اکنون همان کلاس را برای مدیریت فایل‌های صوتی (`sf::SoundBuffer`) نیز به‌کار بگیرید – این دقیقاً همان سناریویی است که آن قالب برایش طراحی شده بود.

۴.۳ کنترل صدا از تنظیمات

میزان صدا باید از طریق منوی تنظیمات (بخش ۵) قابل تغییر باشد و این تغییر باید بلافاصله روی هم موسیقی در حال پخش و هم جلوه‌های صوتی بعدی اثر بگذارد.

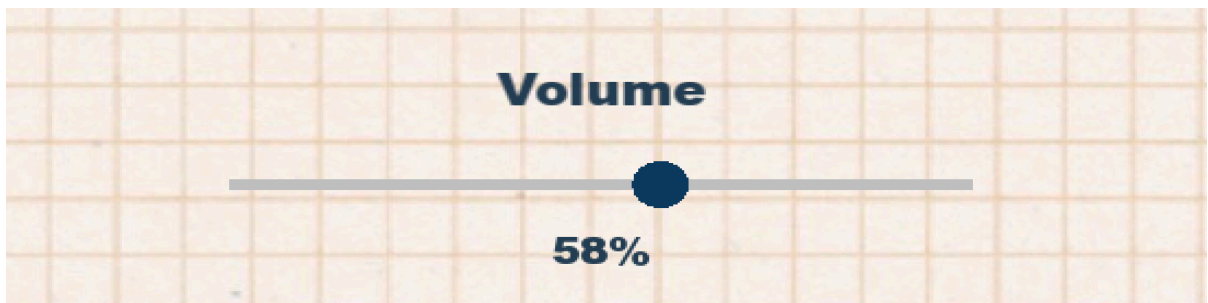


۵. منوی تنظیمات (Settings)

یک وضعیت کاملاً جدید به بازی اضافه می‌شود: **تنظیمات**. این صفحه فقط از طریق دکمه‌ای در **منوی اصلی** در دسترس است و با دکمه‌ی «بازگشت» به منو بازمی‌گردد. تنظیمات را **نمی‌توان** در حین گیم‌پلی باز کرد. موارد زیر باید در صفحه تنظیمات پیاده شوند (تغییرات اعمال شده روی تنظیمات باید برای دفعات بعدی که دوباره برنامه باز میشود ذخیره شوند):

۵.۱ نوار کنترل صدا

یک نوار افقی برای تنظیم میزان صدا (بازهی ۰ تا ۱۰۰) پیاده‌سازی کنید که بر روی تمام صداهای بازی کنترل داشته باشد.

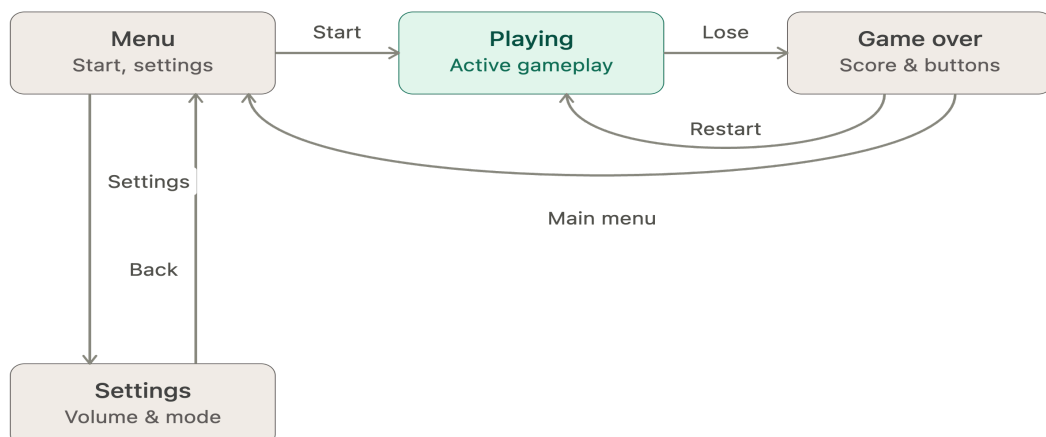


۵.۲ انتخاب سطح دشواری

سه گزینه‌ی قابل‌کلیک برای انتخاب سطح دشواری در نظر بگیرید:

سطح	اثر
Easy	هیولاهای کم‌جان‌تر، نرخ آتش سریع‌تر، بدون حفره
Medium	هیولاهای پرجان‌تر، سکوهای متحرک سریع‌تر، بدون حفره
Hard	مانند متوسط، به‌علاوه‌ی فعال‌شدن حفره‌ها

۵.۳ نمودار وضعیت‌های بازی



Teal = active gameplay, gray = menu / UI screens



۶. حفره‌ها (Holes)

حفره‌ها تله‌های ثابتی هستند که به محض تماس با بازیکن، او را به آرامی به درون خود می‌مکند. این قابلیت فقط در سطح دشواری سخت (Hard) فعال است و بازی را در بالاترین سطح چالش برانگیزتر می‌کند.

۶.۱ ظاهر و تولید

- حفره‌ها در دو اندازه‌ی کوچک و بزرگ وجود دارند که به صورت تصادفی هنگام تولید انتخاب می‌شوند.
- حفره‌ها، مانند هیولاها، در کنار برخی سکوها‌ی تازه تولید شده و با احتمال محدود ظاهر می‌شوند.
- موقعیت حفره باید طوری انتخاب شود که با هیچ سکو، هیولا یا حفره‌ی دیگری هم‌پوشانی یا تماس نداشته باشد، و هرگز تنها مسیر عبور را مسدود نکند.

۶.۲ لحظه‌ی مکش

به محض برخورد بازیکن با یک حفره، رفتار زیر باید رخ دهد:

۱. کل بازی متوقف می‌شود: اسکروِل دنیا، حرکت سکوها‌ی متحرک، حرکت هیولاها و گلوله‌ها – همه چیز باید بی‌حرکت بماند. تنها چیزی که در این لحظه در حال حرکت است، خود بازیکن است.
۲. بازیکن به آرامی، طی یک بازه‌ی زمانی کوتاه و ثابت، کوچک می‌شود و هم‌زمان به سمت دقیقاً مرکز حفره کشیده می‌شود.
۳. وقتی اندازه‌ی بازیکن به صفر برسد، بازی به وضعیت پایان بازی می‌رود.

⚠ اولویت حفره بر هر رویداد دیگر (نکته‌ی حیاتی)

فرض کنید بازیکن روی یک فنر فرود می‌آید، اما در همان لحظه با یک حفره نیز برخورد دارد (مثلاً حفره درست کنار سکوی دارای فنر قرار گرفته). در این حالت، باید بررسی برخورد با حفره پیش از بررسی فنر (و پیش از بررسی سکوها و هیولاها) انجام شود.



۷. نکات تکمیلی

۷.۱ تداخل اشیاء و تضمین مسیر

قانون «عدم هم‌پوشانی سکوها» در فاز اول، اکنون باید به تمام انواع اشیاء دنیای بازی تعمیم یابد. هیچ دو شیئی از میان سکو، هیولا و حفره – از هر ترکیبی – نباید با یکدیگر هم‌پوشانی یا تماس داشته باشند.

📌 یک تابع بررسی مشترک

به‌جای نوشتن بررسی هم‌پوشانی جداگانه برای هر ترکیب، یک تابع عمومی بنویسید که مکان پیشنهادی و ابعاد آن را می‌گیرد و در برابر همه‌ی سکوها، هیولاها و حفره‌های فعال بررسی می‌کند. تمامی توابع تولید سکو، هیولا و حفره باید از همین یک تابع استفاده کنند.

⚠️ تضمین دسترسی‌پذیری هنوز برقرار است

الزام «بازیکن همیشه باید راهی برای بالا رفتن داشته باشد» از فاز اول کماکان پابرجاست. هیولاها و حفره‌ها باید موانع جانبی باشند، نه سدهای عبور؛ هرگز نباید طوری قرار بگیرند که تنها سکوی قابل‌دسترس را از هر زاویه غیرقابل‌فرود کنند.

📌 تولید هیولا/حفره

دقت کنید که نحوه تولید هیولا/حفره نباید به شکلی باشد که در لحظه شروع بازی جلوی بازیکن ظاهر شوند و باعث باخت شوند. (بعد از یک مدتی از شروع بازی شروع به تولید هیولا و حفره کنید)

۷.۲ امتیاز جداگانه برای هر سطح دشواری

از آنجا که سطوح مختلف دشواری، تجربه‌های کاملاً متفاوتی از بازی ارائه می‌دهند، مقایسه‌ی مستقیم امتیازها میان سطوح منصفانه نیست. بنابراین:

- باید سه بالاترین امتیاز مستقل نگهداری شود: یکی برای آسان، یکی برای متوسط و یکی برای سخت.
- هر سه مقدار باید روی دیسک ذخیره و در اجرای بعدی بازیابی شوند.
- در منو و صفحه‌ی پایان بازی، بالاترین امتیاز سطح دشواری فعلی باید نمایش داده شود.



۸. الزامات، نکات پیاده‌سازی و تحویل

الزامات فاز اول همچنان برقرار هستند و باید در سراسر کد جدید نیز رعایت شوند:

۱. رعایت اصول شی‌گرایی: هیولا، حفره و گلوله نیز باید مانند سایر اشیاء بازی در قالب کلاس‌های مجزا با سطح دسترسی مناسب پیاده‌سازی شوند.
۲. بازاستفاده از قالب‌ها (Templates): کلاسی جنریک `ResourceManager<T>` فاز اول را برای مدیریت فایل‌های صوتی نیز گسترش دهید (به بخش ۴.۲ مراجعه کنید).
۳. تبدیل نوع امن: در صورت نیاز به تشخیص نوع اشیاء، از `static_cast` / `dynamic_cast` استفاده کنید.
۴. مدیریت حافظه: با اضافه‌شدن هیولا، گلوله و حفره، ریسک نشت حافظه بیشتر شده است؛ هر شیئی که با `new` ساخته می‌شود باید در جای مناسب `delete` شود – به‌ویژه گلوله‌های خارج از صفحه.
۵. ساختار چندفایلی: کلاس‌های جدید نیز باید به همان شکل `.hpp` / `.cpp` در پوشه‌های `include/` و `src/` سازماندهی شوند.
۶. رعایت اصول Clean Code در تمام کد جدید الزامی است.

📌 فایل‌های جدید مورد نیاز

علاوه بر فایل `highscore.txt` فاز اول (که اکنون باید سه مقدار را نگه دارد)، یک فایل تنظیمات برای ذخیره‌ی میزان صدا و سطح دشواری، و پوشه‌ی `sounds/` برای فایل‌های صوتی به پروژه اضافه می‌شود.

```
simple-doodle-jump/
├── assets/           ← تصاویر بازی (از پیش داده‌شده، شامل هیولا و حفره)
├── sounds/           ← فایل‌های صوتی (از پیش داده‌شده)
├── fonts/            ← فونت بازی (از پیش داده‌شده)
├── include/          ← سازماندهی داخلی با شما - C++ (.hpp)
├── src/              ← سازماندهی داخلی با شما - C++ (.cpp)
├── highscore.txt      ← سه بالاترین امتیاز (آسان/متوسط/سخت)
├── settings.txt       ← میزان صدا و سطح دشواری ذخیره‌شده
└── makefile
```

موفق و سلامت باشید